“Arcade Claw Machine Simulator”

Introduction :

Arcade claw machines are interactive entertainment systems where players control a mechanical claw to grab prizes & drop them into a collection chute. This project recreates that experience as a real-time interactive 3D graphics application.

The simulator features a fully playable claw machine at the center of an arcade environment surrounded by ten decorative machines for added depth. The entire scene is built procedurally using cube & sphere primitives with transformations, incorporating a movable camera, multisource Phong lighting, an articulated claw, gravity based prize movement & a real-time scoring system. By combining core computer graphics concepts with interactive gameplay, the project delivers a functional 3D claw machine simulation.

Objectives :

The main objectives of this project are:

- ➢ Develop an interactive 3D claw machine simulator using OpenGL 3.3 & GLSL shaders.
- ➢ Demonstrate translation, rotation & scaling through transformation matrices.
- ➢ Implement a controllable camera with perspective projection.
- ➢ Animate the claw & mechanical parts using hierarchical transformations.
- ➢ Implement point lights & a dynamic spotlight with per pixel Phong shading.
- ➢ Apply varied material colors & shininess values for visual realism.
- ➢ Provide keyboard controls for claw operation, camera lighting & room rotation.
- ➢ Implement prize grabbing, gravity based falling, chute detection & real-time scoring.
- ➢ Build the arcade environment using reusable cube & sphere primitives.

Tools & Technologies :

Category	Tool / Technology
Programming Language	C++11
Graphics API	OpenGL 3.3 Core Profile
Shading Language	GLSL 330 Core

Window & Input API	GLFW 3
Function Loader	GLAD
Mathematics Library	GLM
Development Environment	Visual Studio

Theoretical Background :

Three dimensional affine transformations are used to position objects in a virtual 3D environment. Translation, rotation & non-uniform scaling matrices change the position, direction & size of object vertices to place them correctly in the world. Hierarchical modeling organizes objects in parent-child relationships, where each child is transformed relative to its parent. This allows the main carriage to move while the attached cable, hub, claw fingers & grabbed prizes automatically follow it.

Viewing & projection transformations control how the 3D scene is seen through a camera. The viewing transformation creates a coordinate system based on the camera's position & viewing direction, then adjusts the scene according to the camera. Perspective projection converts the 3D scene into a 2D view, making distant objects appear smaller than nearby objects. This creates a more natural sense of depth in the scene.

Lighting & shading are used to make objects appear more realistic by simulating how light interacts with their surfaces. The Phong lighting model uses three main components: ambient, diffuse & specular light. Ambient light provides basic overall brightness. Diffuse light depends on the angle between the light & the surface, creating normal surface illumination. Specular light produces bright highlights on shiny materials. Light attenuation reduces the brightness as the distance between the light source & the object increases.

System Design :

Scene Layout & Environment :

The arcade environment consists of a large hall centered around an interactive, full-scale claw machine. The floor features a procedurally generated two-tone checkered pattern computed within the fragment shader. Flanking the main machine are ten decorative claw

machines, five along the left corridor & five along the right scaled down & styled with distinct color themes to give the space realistic arcade proportions.

The Playable Machine & Gameplay :

- ➢ The machine features a blue cabinet with silver trim, a control console with a joystick & red button, a gray prize bed with colored spheres, a hinged trapdoor, a retrieval flap, four support posts, an overhead canopy, a gantry rail, a sliding trolley, a hanging cable & a three finger articulated claw.
- ➢ The claw starts above the prize area in an open position. The player moves it horizontally, lowers it toward a prize & closes the fingers to grab a ball.
- ➢ A successful prize remains attached to the claw. Pressing Enter starts automatic delivery, moving the claw to the chute, opening the fingers & releasing the prize.
- ➢ The released prize falls under gravity & is counted as collected when it reaches the chute.

Lighting Design :

The simulator uses three lights: a front showcase point light, an interior canopy point light & a claw spotlight for focused illumination. Each light has independent keyboard controls, while the spotlight follows the moving claw & the front light changes color based on the room's rotation.

Controls :

Key	Function
Q / W	Move camera forward / backward
E / R	Move camera left / right
T / Y	Rotate room around Y-axis
U / I	Rotate room around X-axis
O / P	Rotate room around Z-axis
A	Reset camera & room rotation
1	Toggle front showcase light

2	Toggle canopy light
3	Toggle claw spotlight
Arrow Keys / S, D, F, G	Move claw
H / J	Lower / raise claw
Space	Open / close claw
Enter	Start automatic prize delivery
Esc	Exit

Animations Used :

- The three claw fingers smoothly rotate between open & closed positions.
- The claw moves horizontally & vertically while the cable extends & retracts.
- A finite state machine controls the automated delivery sequence.
- The chute door rotates open during prize delivery & closes afterward.
- Released prizes fall under gravity until they settle on the bed or enter the chute.
- A grabbed prize follows the claw until it is released.
- The entire arcade hall rotates around multiple axes, with corresponding changes in light color.
- The score updates when a prize successfully enters the chute.

Implementation :

The application starts by configuring GLFW to create a high resolution display context, binding GLAD to access core OpenGL functions & enabling depth testing to resolve object occlusion. Geometric models are generated procedurally: a shared vertex buffer stores 3D positions & surface normal vectors for a standard unit cube, while an algorithmic sphere generator calculates triangle meshes for the prize balls & joystick knobs. Model transformation matrices position, rotate & scale these primitives to construct the complete arcade scene.

Hierarchical transformations control the multi-part claw mechanism. The overhead trolley acts as the parent reference for the cable & central hub, while the three claw fingers are

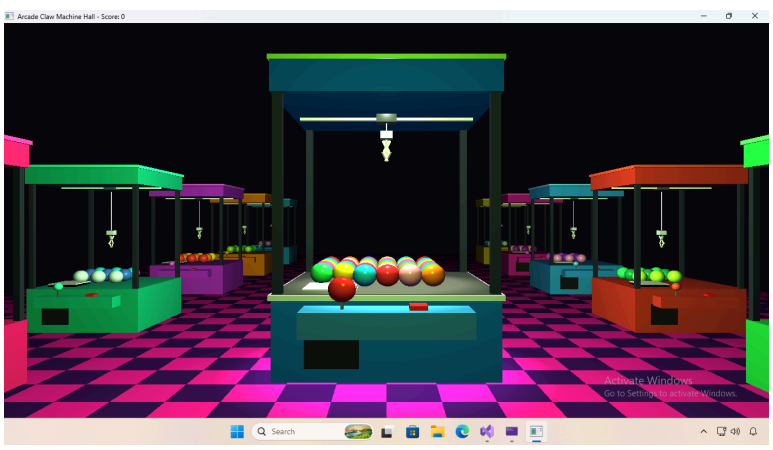
positioned around the hub at equal angular offsets. Each finger contains an upper & lower segment that rotates around local hinge axes, allowing the complete assembly to move while maintaining its structure.

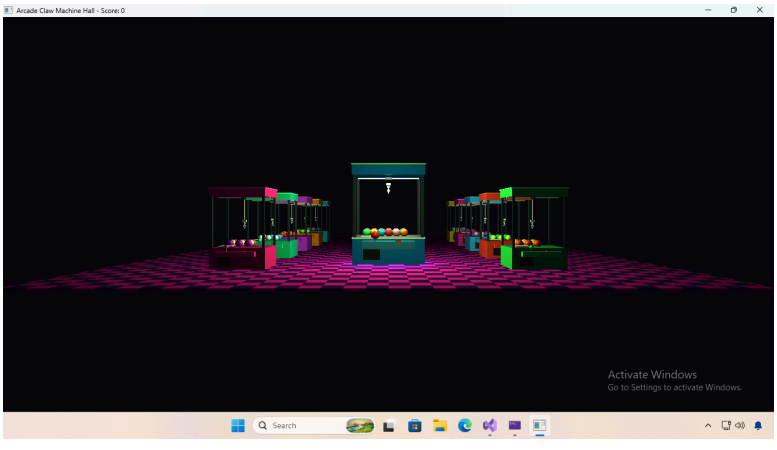
Lighting is computed per pixel using GLSL shaders running the Phong illumination model. Two point lights provide ambient, diffuse & specular illumination with distance based attenuation. A dynamic spotlight is attached directly to the moving claw hub, aiming downward & softening along its outer edges. As the player rotates the entire room, the light positions & orientations transform alongside the geometry & their diffuse tints shift dynamically with the rotation angle.

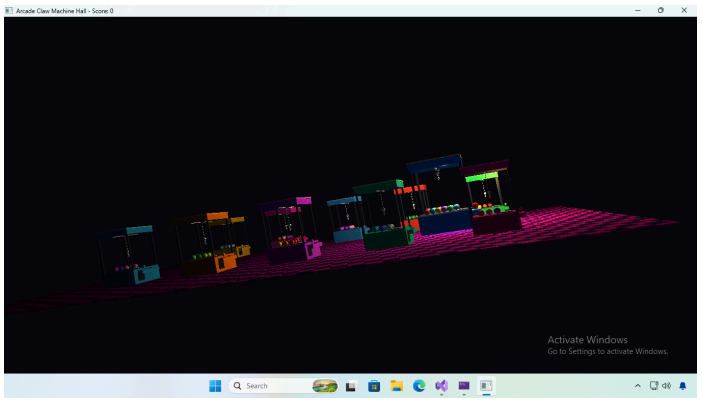
The gameplay loop tracks claw movement, collision zones & ball physics. The player can navigate the crane within safety boundaries, lower it toward the prize bed & close the grip. If the closed claw is lowered sufficiently close to an available prize, that sphere locks to the claw's cradle point. Pressing the activation key starts an automated sequence: the claw lifts the prize to maximum clearance, translates over the chute, opens its fingers & returns to the center. Once released, prizes fall under simulated gravity. Balls that clear the chute opening drop through the trapdoor into the collection bin, updating the score in the window title.

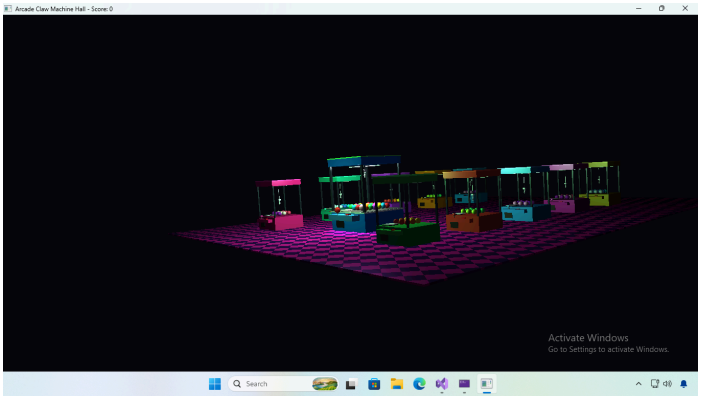
Results & Screenshots :

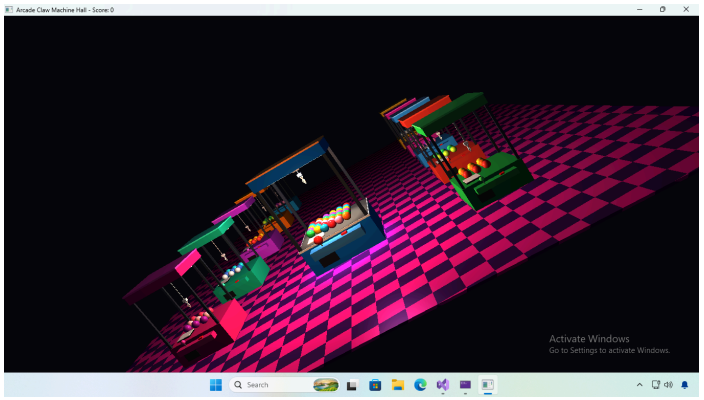
The completed application produces a responsive, real-time 3D arcade environment. The central machine responds to player input through working claw, trolley & cable movements, while the surrounding machines provide additional visual depth. The lighting controls demonstrate the different effects of the point lights & spotlight. The automated delivery sequence moves captured prizes to the collection chute, while successful deliveries are reflected in the live score counter.

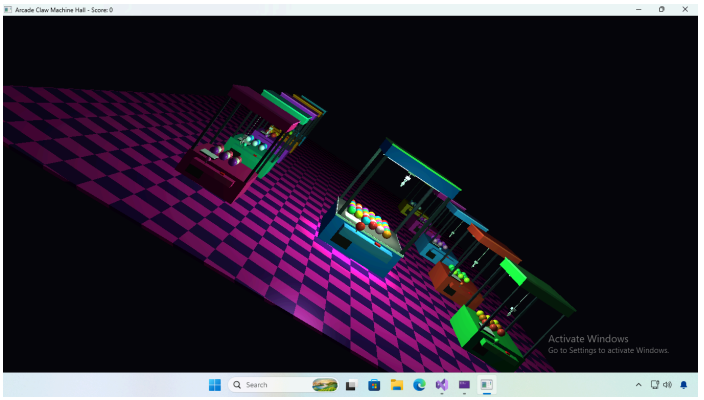


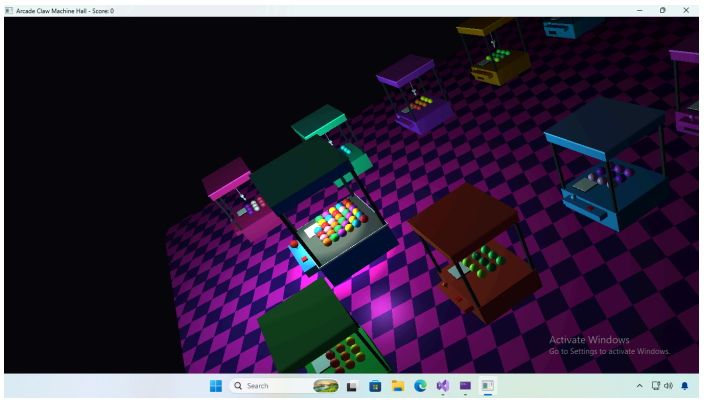


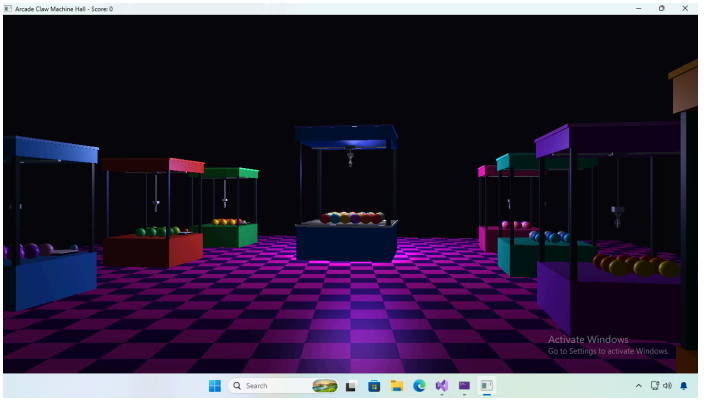

Arcade Claw Machine Hall - Score: 0
Activate Windows
Go to Settings to activate Windows.

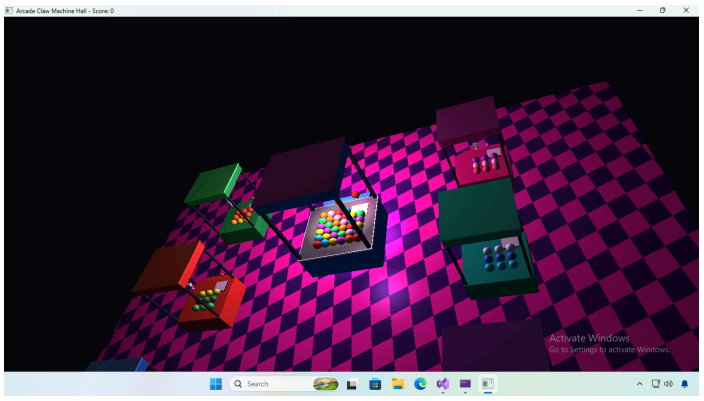

Arcade Claw Machine Hall - Score: 0
Activate Windows
Go to Settings to activate Windows.

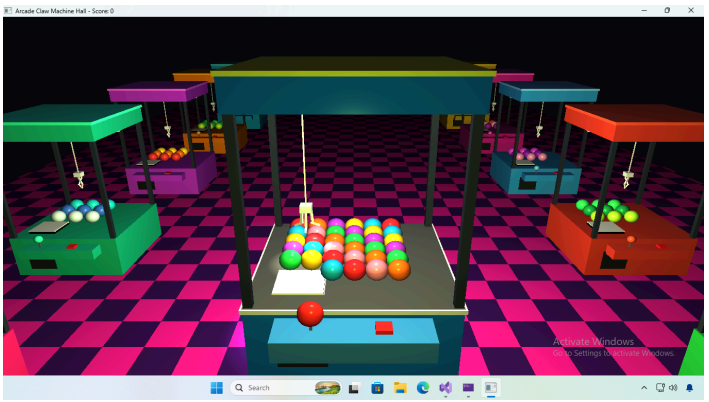

Arcade Claw Machine Hall - Score: 0
Activate Windows
Go to Settings to activate Windows.

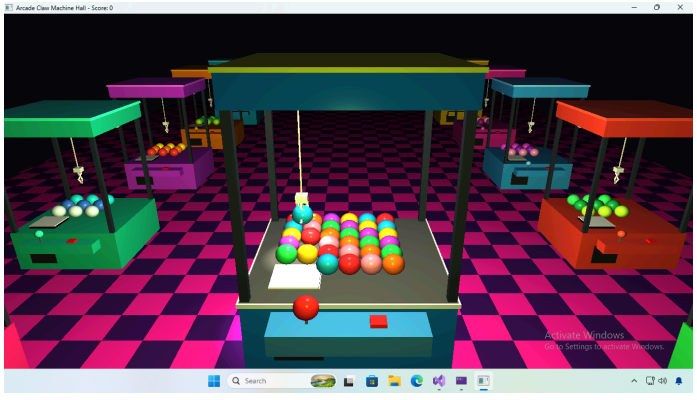

Arcade Claw Machine Hall - Score: 0
Activate Windows
Go to Settings to activate Windows.

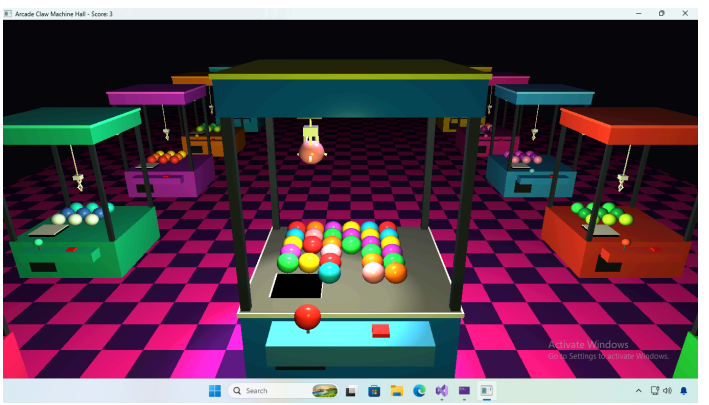

Arcade Claw Machine Hall - Score: 0
Activate Windows
Go to Settings to activate Windows.

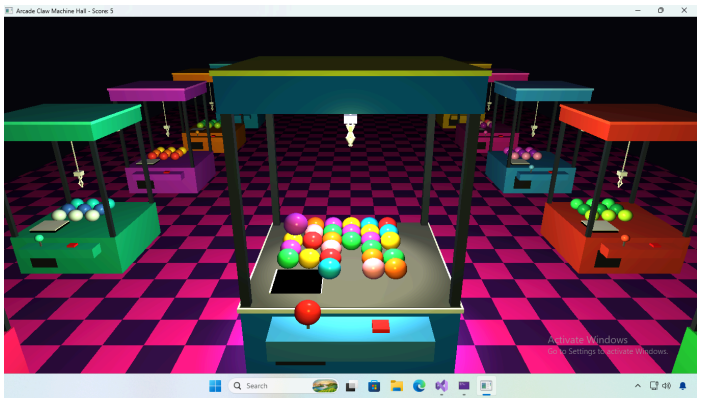
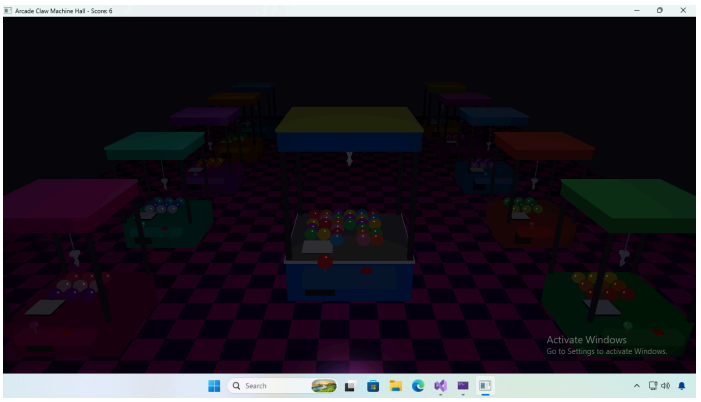
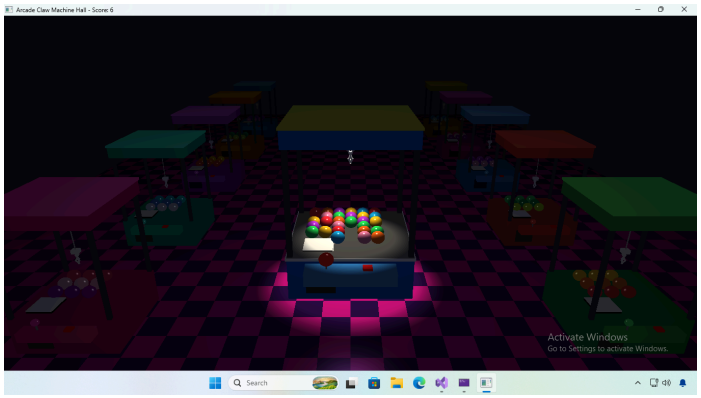
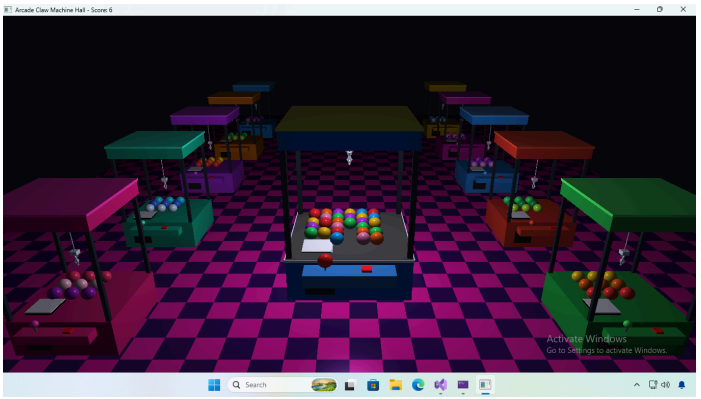
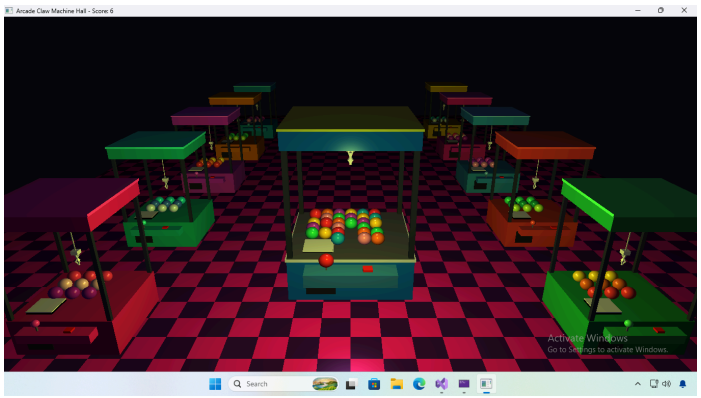
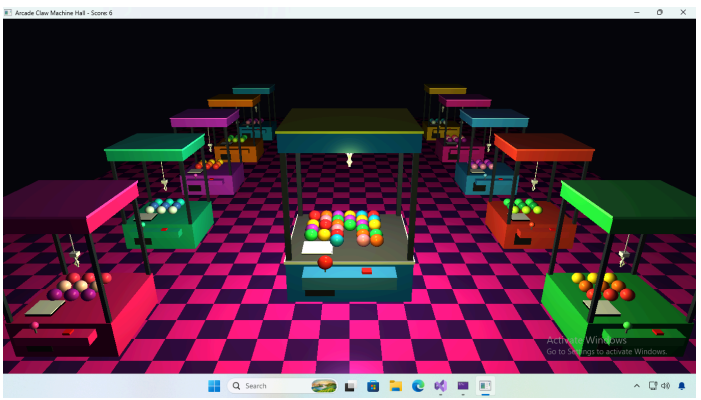
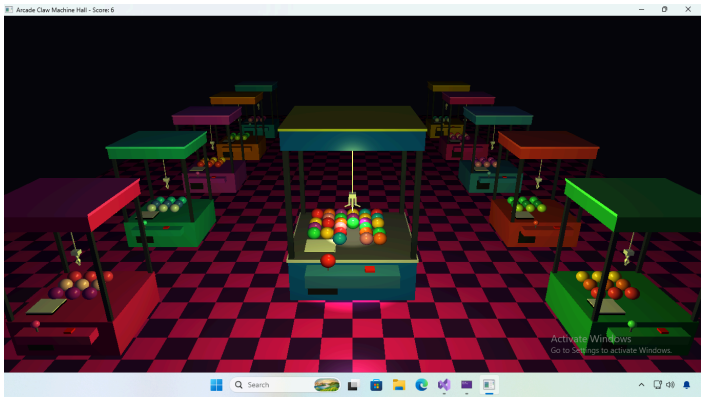
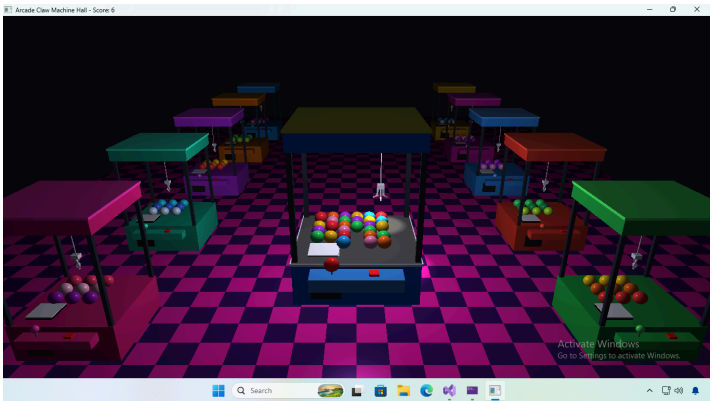

Arcade Claw Machine Hall - Score: 0
Activate Windows
Go to Settings to activate Windows.


Arcade Claw Machine Hall - Score: 0
Activate Windows
Go to Settings to activate Windows.


Arcade Claw Machine Hall - Score: 0
Activate Windows
Go to Settings to activate Windows.


Arcade Claw Machine Hall - Score: 0
Activate Windows
Go to Settings to activate Windows.


Arcade Claw Machine Hall - Score: 3
Activate Windows
Go to Settings to activate Windows.

Discussion :

Building the scene from primitive shapes demonstrates how simple geometric models can

be combined to create detailed 3D objects. Cubes were used for the cabinet, gantry, control panel & other structural elements, while spheres were used for prizes & joystick knobs. This approach kept the geometry simple while allowing complex objects to be created through transformations.

Hierarchical modeling was important for maintaining the structure of the moving crane assembly. Parent-child relationships allowed the cable, hub, fingers & captured prize to move together naturally. This also simplified the automated delivery process because moving the main crane automatically moved its attached components.

The combination of point lights & a spotlight demonstrated how different light sources affect the appearance of a 3D scene. The point lights provide general illumination, while the spotlight focuses attention on the active prize area. Attaching the spotlight to the claw also provides visual feedback about its position during gameplay.

Limitations :

The current implementation has several limitations:

- ➢ No real-time shadow mapping; shadows are represented using flat geometric shadow blobs.
- ➢ Prize physics are simplified, with no ball-to-ball collisions or stacking behavior.
- ➢ The ten background claw machines are static decorative elements & are not playable.
- ➢ No mouse interaction.
- ➢ No audio effects or background music are implemented.
- ➢ No 2D texture mapping; objects mainly use material colors & procedural patterns.
- ➢ No transparency or realistic glass reflections are implemented.
- ➢ Lighting configuration is fixed within the program & shader.
- ➢ The perspective projection uses fixed initial dimensions despite supporting viewport resizing.
- ➢ The spotlight follows room transformation but does not independently track claw orientation.
- ➢ Manual claw control is disabled while the automated delivery sequence is active.
- ➢ Some supporting files, such as alternative camera & Gouraud shading resources, are not used in the active rendering pipeline.

Conclusion :

The project provides a functional arcade environment that applies fundamental computer graphics concepts in an interactive setting. Future improvements could include real-time shadows, texture mapping, audio effects, improved prize physics & mouse based interaction.

References :

1. de Vries, J. *Learn OpenGL: An Introduction to Computer Graphics*. https://learnopengl.com
2. de Vries, J. *Learn OpenGL — Coordinate Systems*. https://learnopengl.com/Getting-started/Coordinate-Systems
3. Khronos Group. *OpenGL 3.3 Core Profile Specification and Documentation*. https://www.khronos.org/opengl/
4. Khronos Group. *OpenGL Official Wiki*. https://wikis.khronos.org/opengl/Main_Page
5. G-Truc Creation. *OpenGL Mathematics (GLM) Library*. https://github.com/g-truc/glm
6. Dav1d. *GLAD: Multi-Language GL/GLES/EGL/GLX/WGL Loader-Generator*. https://glad.dav1d.de
7. Hasan, N. *Course Supporting Headers: camera.h, basic_camera.h, pointLight.h, shader.h, sphere.h*.